1. Problem

Phishing remains the most common entry vector in real-world breaches: a single stolen university or corporate credential unlocks the entire internal network. Attackers rarely need novel exploits - they need a login page that **looks** like the one employees already trust.

Three properties of modern phishing make URL blacklists structurally insufficient. First, **blacklists are always late**: phishing URLs are cheap to rotate, and the median malicious page is only live for hours. Second, **targeted phishing** - a fake unified-authentication page for a specific university, or a fake corporate email/OA login - is built in minutes and **never enters any public blacklist**. Third, **cloud-based anti-phishing services** ask users to upload the URL or even a screenshot, which is equivalent to handing over their browsing history.

- University unified identity-auth, academic-system, enterprise-mail and OA login pages are the top phishing targets (from the original Chinese proposal) — such pages are the highest-value targets in the workplace.

We need a detector that **sees pages the way humans do** - visually recognizing that a page impersonates a trusted brand, in real time, on the user's own device, without any screenshot leaving the machine.

2. Solution

2.1 The three-condition rule

A page is phishing **only if three conditions hold simultaneously**: it impersonates a known brand, its current domain does not belong to that brand, and it asks for credentials. This rule - shared in spirit by Phishpedia, PhishIntention and PhishVLM - is simple, explainable, and keeps false positives near zero on legitimate login pages.

Scenario	Brand spoofed	Domain mis-match	Asks creden-tials	Verdict
Real Alipay login page	**Yes**	No	Yes	Allow
News page introducing Alipay	Yes	Yes	No	Allow
Ordinary forum login	No	-	Yes	Allow
Fake Alipay login page	**Yes**	**Yes**	**Yes**	Block

Three-condition verdict logic. All three must hold for interception; the warning overlay explains exactly which ones fired.

The extension proves condition 3 directly from the DOM (presence of `input[type=password]` and where the form submits), instead of trusting a model to guess it from pixels - this is the single biggest recall lever over the original pipelines.

2.2 Multi-stage funnel

Cheap checks run first; expensive models handle only the few hard cases. Every page enters the funnel and exits at the earliest confident stage, so the accelerator is used where it pays off.

1. **L0 Domain whitelist + LRU cache** (under 1 ms). Known-good domains short-circuit before any model loads; per-URL results are cached.
2. **L1 Structure detection** (Faster R-CNN today, YOLO on Hailo). Locates logos, input boxes and login buttons in the screenshot.
3. **L2 Logo brand matching**. Cropped logos are embedded and compared against the local brand library (Siamese / MobileNet).
4. **L3 Credential-asking check** (1 ms, rules on DOM signals). Password box present, login form, form action leaving the page's own domain.
5. **L4 Local VLM review** (Qwen2-VL-2B, uncertain cases only). Answers: which brand is impersonated, does it ask for credentials, output strict JSON.

> **Decision = block / allow / uncertain. Blocks show an explainable overlay: spoofed brand name, current vs legitimate domain, back and I-confirm-and-continue buttons. Uncertain never blocks - it warns the user to be cautious.**

3. Architecture

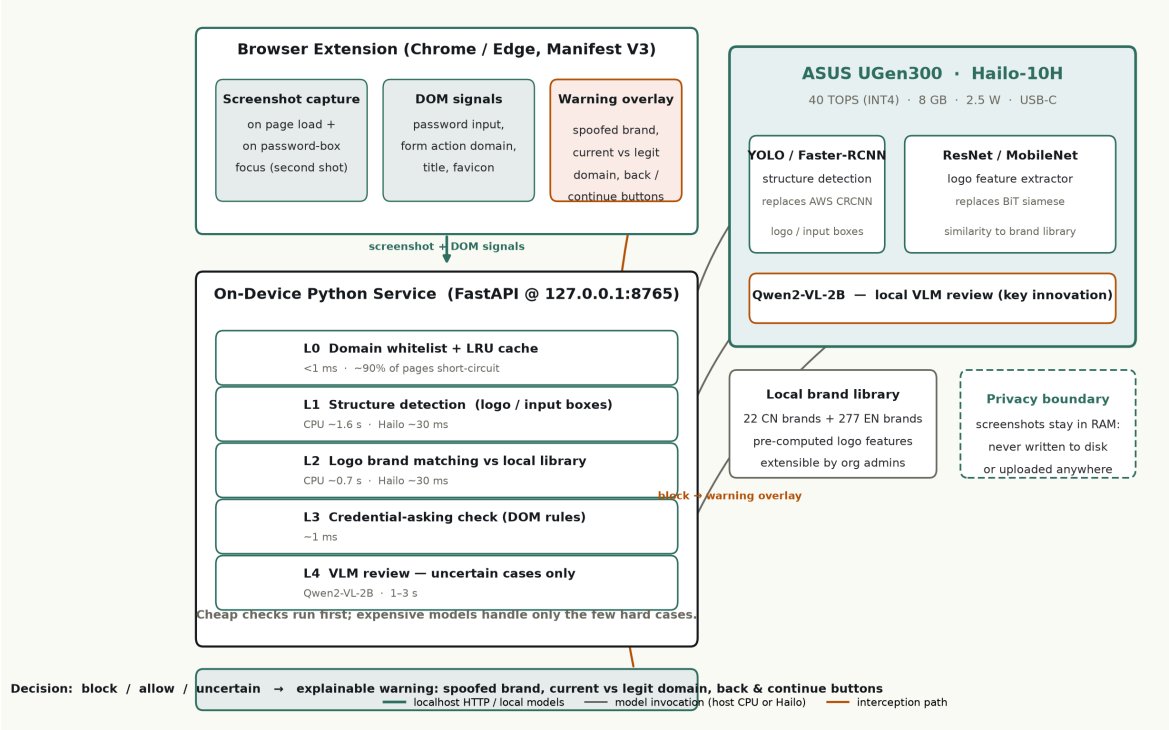


Figure 1: End-to-end architecture. A Manifest V3 browser extension captures screenshots and DOM signals; the on-device FastAPI service runs the five-stage funnel; models run on host CPU now and on the ASUS UGen300 with Hailo-10H in the final round. Everything stays inside the privacy boundary (localhost:8765).

The **browser extension** (Chrome/Edge, Manifest V3) captures a full-page screenshot after load and a second screenshot when the user focuses a password box - the moment with the highest interception value, since many phishing pages show the login form only on interaction. It reads DOM signals for free: password input presence, form action domains, page title and favicon. When a block is returned, it overlays a full-screen warning via Shadow DOM without touching the page's own styles.

The **on-device Python service** (FastAPI, 127.0.0.1:8765) implements the funnel, keeps an event log (JSON Lines, SIEM-ready), and manages the three model families that share the same Hailo chip: YOLO/Faster R-CNN detection, ResNet/MobileNet logo embedding, and Qwen2-VL-2B for VLM review. The **local brand library** holds per-brand meta (name, legitimate domains, logo variants, pre-computed feature vectors) and is extensible by organization admins - their own login pages become protected entries.

The original open-source pipelines were reproduced end-to-end on CPU first (Phishpedia, Phish-Intention and PhishVLM's visual backbone, see documents on GitHub). PhishVLM's LLM review, however, is a **closed cloud API** (gpt-4o-mini + Google Programmable Search, both needing keys many judges cannot run) - PhishLens Edge replaces it with a local Qwen2-VL-2B, which is the key innovation: the privacy claim holds by construction.

4. Hardware Integration

4.1 Why the ASUS UGen300 (Hailo-10H)

The Hailo-10H gives **40 TOPS (INT4), 8 GB on-chip memory, and a typical power draw of 2.5 W** in a USB-C stick that plugs into any Windows/Linux/Android host. For a workplace security peripheral this combination is decisive: it costs no PCIe slot, needs no driver-level GPU setup, and - unlike an NVIDIA card - adds no meaningful power budget or fan noise to a laptop.

- **Same-chip multi-model scheduling**: detection, embedding and VLM are three model families that share one Hailo runtime; the funnel design is exactly what HailoRT's multi-context execution is built for.
- **2.5 W end-to-end privacy**: inference on the stick, so screenshots never travel over PCIe to a cloud, and the host CPU stays free for the user's actual work.
- **Fixed-input-size note**: Hailo compiles with fixed input shapes; a 1280x800 screenshot scaled to 640 shrinks logos, so we crop the top viewport band (where logos and login forms live) or use tiled detection - both patterns already validated in CPU experiments.

4.2 Model porting plan

Stage	Original project	PhishLens Edge	Where it runs
Structure detection	Faster R-CNN (Detectron 2)	YOLOv8 / YOLOX, compiled to HEF	Hailo-10H
Logo features	BiT ResNet siamese	ResNet50 / MobileNet, compiled to HEF	Hailo-10H
Credential check	CRP classifier + HTML rules	DOM rules (extension reads the page)	Host CPU
VLM review	GPT-4o-mini (cloud)	Qwen2-VL-2B, local	Hailo-10H
OCR	ASTER	Reused, host CPU initially	Host CPU

Every cloud or GPU-heavy component maps to a Hailo-compatible local model or a cheaper rule. Target latency: L0 under 5 ms, L1-L3 under 300 ms, L4 under 3 s; fewer than 10% of pages enter L4.

To cut labeling cost, we use **pseudo-labeling**: run the original detection model on thousands of real screenshots on CPU, spot-check by humans, then train YOLO on the pseudo-labeled set and compile to HEF in the Hailo Dataflow Compiler (x86 Linux, per Hailo requirements). The Qwen2-VL-2B is available in Hailo's GenAI model zoo.

5. Privacy & Ethics

Privacy is not a feature here - it is the product architecture. Screenshots and DOM signals travel only over localhost; screenshots are held in RAM and never written to disk or uploaded anywhere. The event log records only the URL decision-relevant signals, not page content.

- **Local processing by construction**: detection, matching and VLM review all run on the user's device (Hailo stick). No browsing history ever leaves the machine.
- **Responsible test practices**: real phishing pages are crawled only in an isolated headless-browser environment (throwaway user-data dir, no cookies, screenshots discarded); self-built fake Chinese pages are deployed only on localhost for testing and clearly labeled as test fakes in the demo video; the public repository contains no directly usable phishing-page source.
- **Prompt-injection defense**: page text is treated as data, never as instructions; the VLM output is strictly schema-validated, and failures degrade to "uncertain" (warn, do not block).
- **User control**: a blocked page offers back and I-confirm-and-continue buttons; a legitimate page being blocked once is enough for a user to uninstall, which is why L0 whitelisting and the three-condition rule keep false positives at zero.

6. Preliminary Results

6.1 Test setup

All experiments ran on a Windows CPU-only machine (no GPU), through the exact same code path as the HTTP endpoint. Test set: 50 phishing screenshots from OpenPhish, 50 benign login pages of real services, and 3 self-built Chinese fake pages (Alipay, WeChat, ICBC) with embedded official logos.

Split	Count	Source
Phishing	50	OpenPhish feed, quality-filtered, deduplicated
Benign	50	Real login pages (Weibo, Zhihu, Amazon, Steam, Tsinghua, SDU, gov.cn, banks, ...)
Fake CN pages	3	Local HTML clones of Alipay / WeChat / ICBC logins

Test set overview. Brand library: 22 protected Chinese brands + 8 key English brands, plus the 277-brand / 3,064-logo international reference library.

6.2 Detection results

Protocol	Set	block	uncertain	allow	Total
Visual-only	Phishing	0	13	37	50
Visual-only	Benign	0	0	50	50
Visual-only	Fake CN	3	0	0	3
Full-DOM	Phishing	13	0	37	50
Full-DOM	Benign	0	0	50	50
Full-DOM	Fake CN	3	0	0	3

Confusion matrices at decision level, threshold 0.68. Visual-only reports what pure pixel analysis finds; full-DOM adds the real extension signals (password box), converting all 13 visual hits into hard blocks.

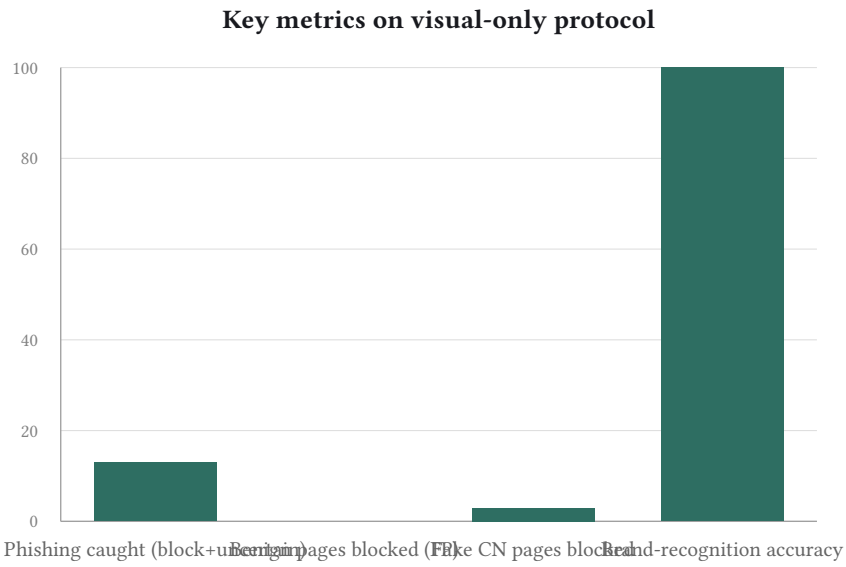


Figure 2: Phishing recall 26% (13/50), false-positive rate 0% (0/50), fake-page interception 3/3, brand-recognition accuracy 13/13 = 100% whenever a brand is fired.

Why the number 26% is honest, and how the funnel fixes it. The visual path only fires when the detector finds a logo region AND it matches a protected brand above 0.68. 37/50 OpenPhish samples miss because (a) the Faster R-CNN finds no logo box on crowded pages, so the fallback top-strip crop's embedding is diluted below threshold, and (b) new-English-brand pages (Coinbase, Ledger, Xfinity, Shopee) have tiny or occluded logos in crawled screenshots. Brand-matching quality itself is high - 13/13 = 100% whenever a logo is found - so logo-region **recall** is the binding constraint, not accuracy.

6.3 Latency and funnel split

Latency below is per-sample, hot (a warm-up sample absorbs the one-shot cold model load of 789 s: detectron2 + Siamese + OCR + logo-feature cache). The second column shows the Hailo target for the final round.

Stage	CPU (current)	Hailo target
L0 whitelist / cache	0 ms	0 ms
L1 logo detection	1.592 s mean	30 ms
L2 brand matching	0.730 s mean	30 ms
L3 DOM check	0 ms	0 ms
L4 VLM review (uncertain only)	hook, 0 ms	1-3 s
Total per page	2.144 s mean	under 300 ms

Measured CPU latency (hot, n=103) and the final-round Hailo targets.

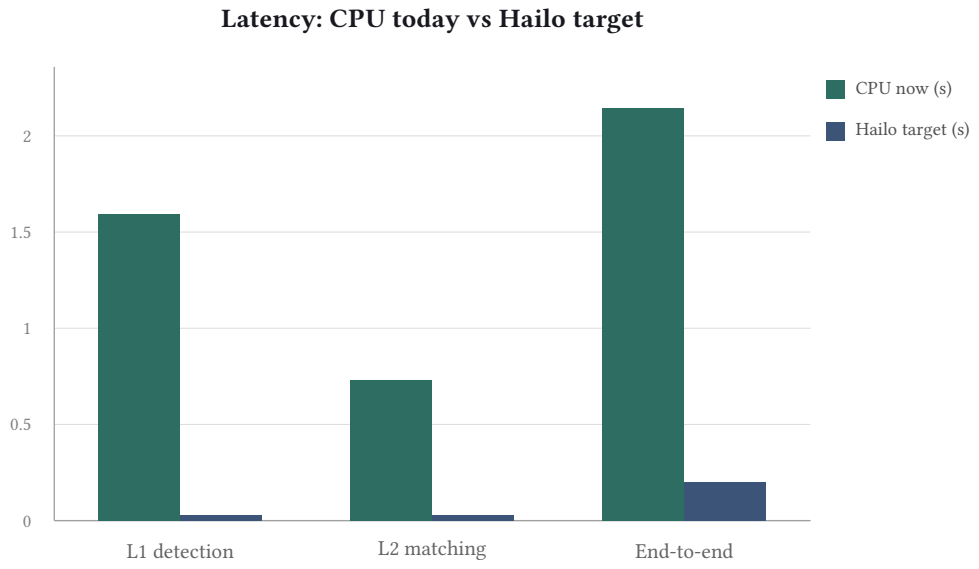


Figure 3: Costly stages move from 1.6 s / 0.73 s on CPU to 30 ms each on Hailo-10H; the interactive path targets below 300 ms.

Funnel split: 95/103 samples took the full visual path (L1-L2); 8 benign samples short-circuited at the L0 whitelist (weibo.com, zhihu.com, amazon.com, tsinghua.edu.cn, sdu.edu.cn, facebook.com, steampowered.com, gov.cn). None of the 50 phishing URLs were whitelisted. 13 + 3 samples reached a block/uncertain decision driven by brand match; every detected page's genuine brand was present in the reference library.

Next round: raise logo-recall with YOLO fine-tuned on pseudo-labeled crops and a headline-text OCR brand-hint path (prototyped as L3 hints), then evaluate 500+500 on Hailo.

7. Business Model

7.1 Target customers

- **Universities**: protect unified authentication, academic-affairs and campus-mail login pages; protect students and staff.
- **SMEs**: protect corporate mail, OA and finance systems without a dedicated security team.
- **Finance & government**: employee terminals anti-phishing where data must not leave the intranet.

7.2 Revenue model

- Hardware one-time sale (UGen300 + extension + admin console), priced per seat for volume deployments.
- Annual subscription for brand-library and model updates (new banks, new domains, re-trained HEFs).
- Go-to-market: pilot at our own campus to vocational schools and universities, then SMEs, then finance/government; ASUS partnership positions it as a security accessory or preload option for business laptops.

7.3 Competitive comparison

	Browser blacklist	Cloud anti-phishing	Open-source (Phishpedia)	PhishLens Edge
Zero-day phishing	No	Partial	Yes	Yes
Targeted (org-specific) phishing	No	No	Known brands only	Yes - custom protected library
Privacy (data stays local)	Good	No - uploads URL /screenshot	Good	Good
Chinese brands	-	Partial	No	Yes - 22 CN brands and growing
Hardware needed	None	None	NVIDIA GPU	2.5 W USB stick
Explainability	Low	Low	Medium	High - overlay explains why

Positioning: the only option that combines zero-day + targeted-phishing detection, full local privacy, Chinese brand coverage and an explainable warning on a 2.5 W stick.

8. Roadmap

The competition delivers the ASUS UGen300 to finalists on 10/26; the final-round development window is 11/6-12/4. Everything here is planned against the hardware arrival date.

Phase	Timeline	Milestone
Qualifying submission	Before 10/14	This brief + demo video + public GitHub (CPU prototype)
Hailo bring-up	10/26-11/5	Receive UGen300; set up x86 Linux compile env; verify HailoRT + Qwen2-VL-2B
Model porting	11/6-11/17	Pseudo-label 1k+ screenshots; train YOLO + MobileNet; compile HEF; switch server to Hailo backend
Product hardening	11/18-11/25	Admin console for custom brands; brand library to 100+; full 500+500 evaluation; power and CPU-utilization tests
Engineering & submission	11/26-12/4	Unit tests, one-click Windows installer, docs; final brief + 10-min video showing the stick live

Final-round plan. The architecture already treats the model backend as replaceable (CPU now, HailoRT later), so this roadmap is de-risked.

Success metrics for the final round: phishing recall >= 90% at 0% false positives on 500+500, brand-recognition accuracy >= 95%, end-to-end latency < 300 ms on Hailo, and a measured host-CPU drop versus the CPU-only baseline - the numbers the brief must prove with a live stick.

9. References & GitHub

[1] Lin, R. et al. Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages. USENIX Security 2021. https://github.com/lindsey98/Phishpedia (CC0-1.0).

[2] Lin, R. et al. PhishIntention: A Semantic-based Approach for Phishing URL Detection. USENIX Security 2022. https://github.com/lindsey98/PhishIntention (CC0-1.0).

[3] code-philia. PhishVLM: Visual language models for phishing detection. 2025. https://github.com/code-philia/PhishVLM (visual backbone reproduced on CPU in this project; its LLM review step is a closed cloud API - gpt-4o-mini + Google PSE - which this project localizes).

[4] Liu, R. et al. Reference-based Phishing Detection without a Pre-defined Reference List. USENIX Security 2024.

[5] Yu et al. Robustness Evaluation of Visual Phishing Detectors. USENIX Security 2025. arXiv:2405.19598 (methodology referenced for robustness testing).

[6] OpenPhish - free real-time phishing URL feed. https://openphish.com/phishing_feeds.html (data source for the 50 phishing screenshots).

[7] ASUS UGen300 specifications: INT4 40 TOPS, 8 GB, Hailo-10H, USB-C, 2.5 W typical. https://www.asus.com/motherboards-components/ai-accelerator/ugen/ugen300-usb-8g/techspec/

[8] hailo-ai. Hailo Model Zoo / GenAI. https://github.com/hailo-ai/hailo_model_zoo_genai (Qwen2-VL-2B availability).

[9] GitHub repository: https://github.com/wushifdvcbavgh11/PhishLens-Edge (public, Apache-2.0). Includes this brief, architecture diagram (docs/architecture.png), evaluation scripts, brand library tooling, extension and server source, README and THIRD_PARTY_LICENSES.